

Improvement of Architecture in Last Level Cache With Non-Volatile Memory

Ziyu Zhu*, Luobin Ni*, Faquan Wang*

Abstract—The increasing memory demands in last-level caches (LLCs) have driven researchers, including Apostolos Kokolis, to propose the Cloak architecture, which replaces traditional SRAM with Non-Volatile Memory (NVM) and incorporates page buffers (PBs) for faster access[1]. However, the relatively low cache hit rate of PBs results in high access latencies due to the high read and write latencies of the NVM.

This paper addresses these limitations by introducing two key improvements to the Cloak architecture: an enhanced prefetching mechanism between PBs and NVM and an optimized replacement policy for PBs. The proposed prefetcher offers more accurate data predictions, while the improved replacement policy enables more efficient management of PB storage. Together, these enhancements aim to reduce access latency and improve overall cache performance. * †

Index Terms—Last-Level Cache (LLC), Non-Volatile Memory (NVM), Page Buffers (PBs), Cloak Architecture, Prefetching, Replacement Policies, High-Performance Computing, Cache Optimization

I. INTRODUCTION

THE increasing demand for high-performance computing has significantly amplified the need for larger Last-Level Caches (LLCs). Traditionally, LLCs are designed using SRAM technology. However, further increasing the capacity of SRAM-based caches introduces several challenges, including substantial area overhead [2], [3], significant leakage power, and limited scalability [4].

As an alternative, Non-Volatile Memory (NVM) has been explored due to its potential for higher capacity and reduced leakage power. Nevertheless, NVM suffers from high read and write latencies, limiting its effectiveness in cache designs. To address these issues, Dr. Kokolis and his research group proposed a novel architecture, Cloak, which mitigates NVM's limitations by incorporating a page buffer and reconfiguring the hardware connections. This innovative design aims to reduce access latency and improve overall cache performance.

Each time a cache is hit in the NVM LLC, the required cache line is pushed to the Page Buffer for future access. The Page Buffers utilize a dynamic replacement strategy that considers access frequency and recency, evicting blocks that have not been accessed for a defined period. However, testing by Dr. Kokolis and his team revealed that only 54% of the

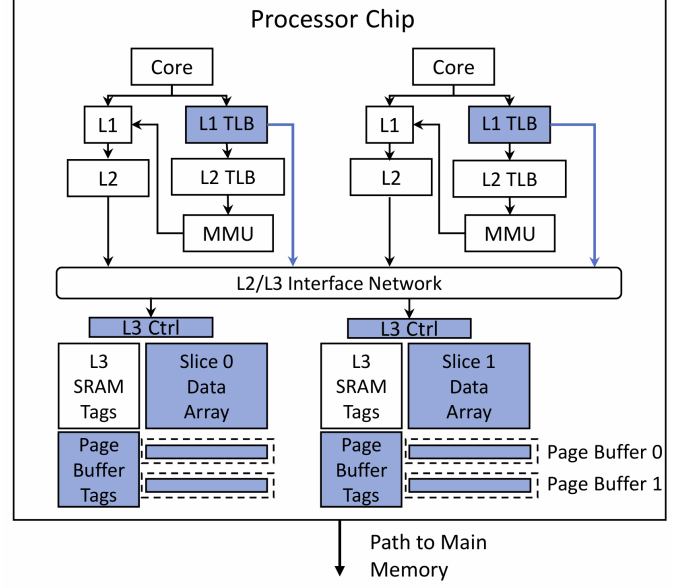


Fig. 1. Cloak architecture, with the new or modified hardware structures and connections highlighted.[1]

cache lines survive in the Page Buffers [1], resulting in high read latencies due to the inherent characteristics of the NVM cache.

To address this issue, the team evaluated four different prefetchers using the *Structural Simulation Toolkit (SST)*[5] and tested three replacement policies with Simics[6]. In these experiments, a 4-core processor was used and the Cloak architecture was implemented with approximately 90% functionality. Due to time constraints, the connection between the L1 TLB and the L2/PB interface network topology connection (as shown in Fig. 1) could not be implemented. The SST framework does not provide sufficient documentation to facilitate modifications to the internal network topology.

This study aims to improve the Cloak architecture by implementing an advanced prefetching mechanism and an optimized replacement policy for page buffers (PBs). These enhancements are designed to increase PB hit rates by approximately 18%, reduce the number of swaps between PBs and non-volatile memory (NVM) by 3.3%, and consequently decrease access latencies by 40%. Additionally, these changes contribute to an increase 2% in Instructions per Cycle (IPC), demonstrating an overall improvement in cache performance. These quantitative enhancements effectively address the high latency issues typically associated with NVM-based last-level caches (LLCs), resulting in a more efficient and responsive

Ziyu Zhu, UNI: zz3176, e-mail: zz3176@columbia.edu.

Luobin Ni, UNI: ln2516, e-mail: ln2516@columbia.edu.

Faquan Wang, UNI: fw2412, e-mail: fw2412@columbia.edu.

*Equal contribution. Listing order is random. Luobin focused on sst benchmark implementation, Ziyu Zhu designed multi-core architecture and prefetcher, Faquan implemented simics simulation.

†Artifacts are available at

<https://github.com/NLBMax/EE6894-Team-Microarchitecture>

cache hierarchy.

A. Cloak Architecture

Cloak is a data center chip architecture developed through collaboration between scholars from the University of Illinois Urbana-Champaign and technology companies, including Nvidia, AMD, and Rivos [1][7]. This architecture addresses the growing demand for larger Last-Level Caches (LLCs) in data centers. Due to the limitations of SRAM in terms of cost, power consumption, and area efficiency, larger LLCs are necessary to handle massive data processing workloads. Although read and write latency is less critical in this context, the proposed architecture substitutes SRAM with Non-Volatile Memory (NVM), which offers significantly enhanced area efficiency and reduced power consumption.[8][9]

Although extensive research has been conducted to mitigate the high read latency of NVM, the authors identified a lack of focus on its high read latency. To address this gap, Cloak introduces a new layer called Page Buffers (PBs), as illustrated in Fig. 1. These PBs act as a fast access buffer to the NVM LLC, holding a set of cache lines from the NVM cache and reducing the latency between the LLC and other cache levels [1].

The promotion of cache lines from a page to a Page Buffer (PB) is illustrated in Fig. 2(a). When a cache hit occurs in the LLC but results in a miss in the PBs, the required cache lines are promoted to the PBs for subsequent access. PBs employ a dynamic replacement policy that evicts blocks that have not been accessed within a specified time frame, ensuring efficient management of the buffer space [1].

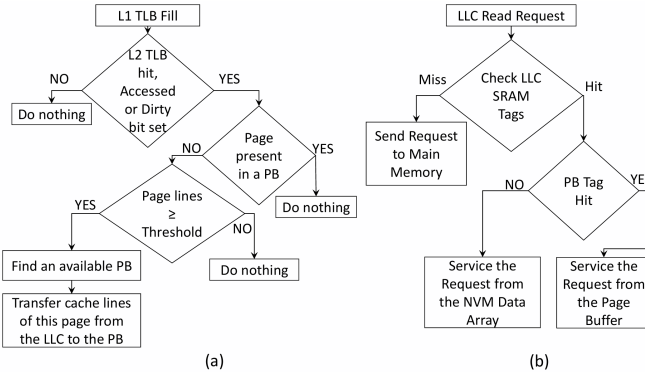


Fig. 2. Control flow diagrams: (a) Promotion of lines of a page to a PB, and (b) Servicing a read request from the LLC [1].

The data control flow of Cloak is illustrated in Fig. 1 and Fig. 2. When a cache miss occurs in the L1 cache, the access request is sent simultaneously to the L2 cache and the LLC, allowing the LLC additional time to search for the requested data. The LLC provides data only when there is a cache miss in the L2 cache but a cache hit in either the Page Buffers (PBs) or the NVM cache. As the PBs are constructed with SRAM, their access latency is significantly lower than that of the NVM cache. However, this creates a critical dependency: the overall access latency of the LLC is heavily influenced by the cache hit rate in the PBs [1].

II. MOTIVATION

Despite the innovative design of the Cloak architecture, empirical evaluations indicated that the Page Buffers (PBs) achieve only a 54% cache line survival rate [1] (Fig. 3). This suboptimal performance is mainly attributable to the high read and write latencies of NVM, which require frequent data exchanges between PBs and NVM, thereby incurring significant access delays.[10]

The low PB hit rate undermines the primary objective of reducing the NVM access frequency, resulting in lower cache performance and increased latency. To address this, there is a compelling need to improve both the pre-fetching mechanisms and the replacement policies that govern PBs. Improved prefetching can enhance the accuracy of data predictions, ensuring that relevant data are readily available in PBs. Currently, an optimized replacement policy can ensure that the most valuable data remains in PBs, maximizing cache utilization, and minimizing unnecessary NVM accesses [1].

Therefore, this study aims to introduce an advanced prefetching mechanism coupled with a hybrid replacement policy to increase PB hit rates and reduce access latencies, thus significantly improving the overall performance of the Cloak architecture [1].

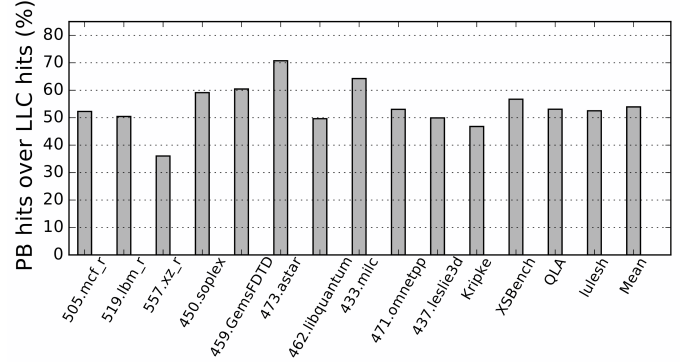


Fig. 3. Percentage of LLC hits that are serviced by PBs [1].

III. CLOAK IMPLEMENTATION

A. 4-Core Cloak Design

The Cloak architecture incorporates a 4-core processor design to optimize the performance of Last-Level Caches (LLCs). The detailed design and improvements are described below.

The architecture in Fig. 4 depicts a multi-core processor system with four CPUs (CPU_0 to CPU_3), each equipped with a private two-level cache hierarchy consisting of L1 Cache (32KB) and L2 Cache (64KB). The CPUs are interconnected through a network layer that connects them to NVM of 16MB capacity each, facilitated by 50KB page buffers for essential data prefetched by the large NVM. The whole architecture has a main memory of 64GB also connected to the network layer.[11][12]

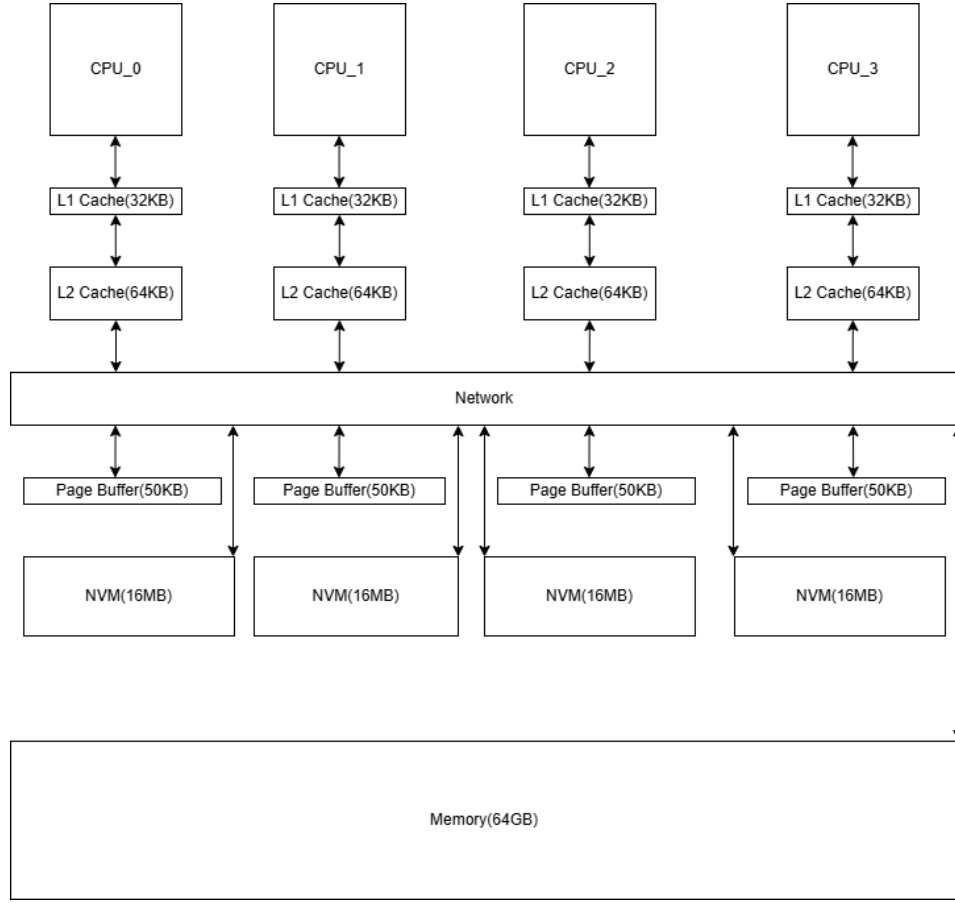


Fig. 4. 4 core architecture

B. Prefetcher Design

The prefetcher is implemented between all levels of caches to accelerate memory access by fetching data from slower memory to faster memory before it is explicitly requested. This prediction mechanism helps the CPU access data more efficiently in the cache, avoiding the latency incurred by searching lower-speed memory or storage.

In the Cloak architecture:

- The **L1 cache** employs a *stride prefetcher*, which detects fixed stride patterns, such as sequential column accesses in a matrix. This enables efficient prefetching for workloads with repetitive and predictable strides.
- The **L2 cache** utilizes a *next-block prefetcher*, predicting accesses to the next one or more blocks based on observed patterns. This prefetcher is particularly effective for workloads involving sequential memory block access, such as streaming data.

The most critical prefetcher in the Cloak architecture operates between the Page Buffers (PBs) and the NVM. The authors evaluated four prefetchers using the Structural Simulation Toolkit (SST):

- 1) **Stride Prefetcher:** Efficiently detects and predicts strides for workloads with regular memory access patterns. [13]
- 2) **Next-Block Prefetcher:** Predicts sequential block accesses for linear memory traversal. [13]

- 3) **Palacharla Prefetcher:** An advanced stride prefetcher[14] that detects strides between consecutive memory accesses and updates stride information for repeated patterns. The main algorithm is

$$\text{target_prefetch_address} = \text{target_address} + (\text{stride_reach} \cdot \text{stride})$$

This calculates the prefetching address by adding a scaled stride to the current target address.[15]

- 4) **Single Exponential Smoothing Prefetcher:** Predicts future memory accesses using a weighted average of past addresses, dynamically adapting to trends in access patterns.[16] Its main algorithm is:

$$\text{Score}_{\text{new}} = \alpha \cdot \text{access_indicator} + (1 - \alpha) \cdot \text{Score}_{\text{previous}}$$

where:

- α : Smoothing factor ($0 < \alpha < 1$).
- *access_indicator*: Indicator variable (1 if the cache line is accessed, 0 otherwise).

C. Replacement Policy

Effective cache management[17] is critical for optimizing LLC performance, especially when integrating NVM and PBs. This study evaluates three replacement policies within the Cloak architecture:

1) Least Recently Used (LRU):

- **Mechanism:** Evicts the cache line that has not been accessed for the longest duration, assuming temporal locality.
- **Implementation:** Each PB cache line is assigned a timestamp upon access. The line with the oldest timestamp is evicted when space is needed.[18]

2) Exponential Smoothing Prediction (ESP):

- **Mechanism:** Assigns a prediction score to each cache line, updated using exponential smoothing to prioritize recently accessed data:
- **Implementation:** Utilizes a priority queue to efficiently track and evict cache lines with the lowest prediction scores.[19]

3) Least Frequently Used (LFU):[19]

- **Mechanism:** Tracks access frequency for each cache line, evicting the one with the lowest frequency count.
- **Implementation:** Employs a min-heap to manage and retrieve the least frequently accessed cache lines.

These policies were implemented and tested within the Simics simulation environment to evaluate their effectiveness in improving PB hit rates and overall LLC performance.

IV. EVALUATION METHODOLOGY

In Dr. Kokolis' paper, his team utilized both the Structural Simulation Toolkit (SST) and Simics to simulate the Cloak architecture. Inspired by their approach, our team attempted to integrate SST with Simics. However, we found no open-source solution for implementing SST within the Simics environment. As a result, we conducted separate simulations on both platforms: SST was used for developing and testing new prefetchers, while Simics was employed for evaluating replacement policies. This division allowed us to leverage the strengths of each system despite the lack of integration.

A. SST Simulation Setup

The Structural Simulation Toolkit (SST) was configured to evaluate the enhanced Cloak architecture. The SST environment with both core and element was used to test the performance of various prefetching mechanisms and replacement policies under diverse workloads. The configuration parameters, including memory hierarchy and prefetching algorithms, were aligned with the Cloak design to ensure simulation accuracy.

In the SST simulation, four different benchmarks were used to evaluate the performance of the Cloak design: GUPS Benchmark, STREAM Benchmark, Stencil3D Benchmark, and Sparse Matrix-Vector Multiplication.

- **GUPS Benchmark:** Commonly used to evaluate random memory access performance in high-performance computing systems.[20]
- **STREAM Benchmark:** Designed to assess a system's ability to handle large datasets that exceed cache size.[21]

- **Stencil3D Benchmark:** Simulates 3D stencil computations, often used in applications like heat diffusion or fluid dynamics.[18]
- **Sparse Matrix-Vector Multiplication:** Models memory access patterns for sparse matrix computations, commonly encountered in scientific and engineering applications.[22]

In the SST simulation, four candidates are evaluated to determine which offers the best performance: the Next Block Prefetcher, the Palacharla Prefetcher, the Stride Prefetcher, and a dynamic prefetcher based on Exponential Smoothing Prediction. Their performance is assessed based on two key metrics: the cache hit rate of the Page Buffers and the simulated time of the benchmarks.

B. Simics Simulation Setup

The Simics simulation environment was meticulously configured to accurately model the enhanced Cloak architecture, integrating Non-Volatile Memory (NVM) and optimized Page Buffers (PBs) with advanced prefetching mechanisms and distinct replacement policies. The following outlines the comprehensive setup process to ensure that the simulation accurately reflects real-world hardware behavior and workload conditions.

1) *System Initialization:* The system initialization for Simics included the following configurations:

- **Simics Version:** 2023.1
- **Operating System:** Customized Linux kernel compatible with Simics.
- **Processor Model:** 4-core multi-threaded processor architecture, representative of typical data center workloads.
- **Memory Configuration:** Sufficient main memory allocated to accommodate large cache sizes and NVM.
- **Scripted Setup:** A primary Python script `simics.py` was developed to automate the initialization of the simulation environment. This script orchestrated the creation and configuration of cache hierarchies, integration of replacement policies, and loading of benchmark applications.

2) *Replacement Policy Integration and Benchmark Loading and Execution:* To evaluate the performance of replacement policies, the following modules were utilized:

- **Replacement Policies:** The `policies.py` module instantiated the appropriate policy class based on the configuration and bound it to the PBs within the cache hierarchy.
- **Benchmark Loading:** The `benchmarks.py` module facilitated the loading and execution of the SPEC CPU 2017 and PARSEC benchmark suites within the Simics environment.

Benchmarks were executed under each replacement policy to evaluate their impact on cache performance, including hit rates, access latencies, and overall throughput.

V. EVALUATION

A. SST Simulation Result

Fig. 5 illustrates the cache performance across different benchmarks for a specific page buffer (Page Buffer 0). All

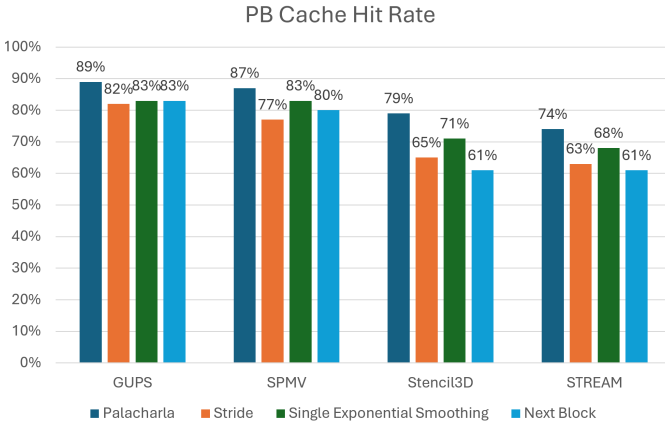


Fig. 5. Page Buffer 0 Cache Hits Misses Rate

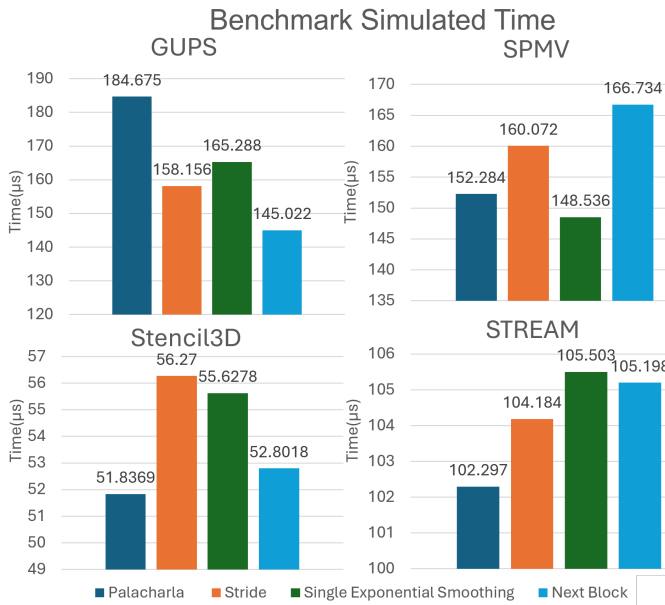


Fig. 6. Benchmark Simulated Time

the other three PBs are having similar results. To examine the whole data, please check in *Artifacts*. Four benchmarks are evaluated: Palacharla, Stride, Single Exponential Smoothing, and Next-Block. Each benchmark measures the hit/miss rate for four distinct workloads: GUPS, SPMV, Stencil3D, and STREAM.

- For the **GUPS** benchmark, the highest cache hit rate is achieved with Palacharla (89%), while the other benchmarks maintain similar hit rates. This demonstrates that all benchmarks perform relatively well for GUPS, with Palacharla offering the highest efficiency.
- In the **SPMV** benchmark, Palacharla again achieves the highest cache hit rate (87%). This indicates that Palacharla exhibits superior caching performance for this workload, with Single Exponential Smoothing also performing strongly.
- For **Stencil3D** benchmark, the hit rates drop significantly compared to the previous workloads. Palacharla maintains a reasonable hit rate of 79%. While others are down to

around 65%. This suggests that the workload's memory access patterns are less favorable for caching, especially for Next Block.

- Lastly, in the **STREAM** benchmark, Palacharla achieves the highest hit rate (74%). STREAM demonstrates less locality, with lower hit rates overall compared to GUPS and SPMV.

Fig. 5 highlights how the cache hit rates vary based on the workload and benchmark, with Palacharla consistently delivering the highest hit rates across all workloads.

Fig. 6 presents the simulation times for the same benchmarks and workloads. The results highlight the relationship between cache efficiency and execution time.

- For the **GUPS** workload, Next Block achieves the lowest execution time (145.022 μs), while GUPS has highest (184.675 μs).
- In the **SPMV** workload, the lowest simulation time is observed with Single Exponential Smoothing (148.536 μs), followed with Palacharla. Next Block has the highest time (166.734 μs).
- For **Stencil3D**, the Palacharla benchmark shows the fastest execution (51.8369 μs), followed by Next Block, while Stride has the highest time (56.27 μs).
- Finally, the **STREAM** benchmark shows minimal variation in execution times across all, with the fastest being Palacharla (102.297 μs) and the slowest being Single Exponential Smoothing (105.503 μs), indicating consistent performance irrespective of cache behavior.

The Palacharla prefetcher has the best performance in most of the scenarios. With highest cache hit rate in all four benchmarks, and three shortest simulated time, Palacharla can be used as the default prefetcher. Next Block has an advantage in GUPS, so it can be a backup plan for the system if high performance streaming data is used. Stride prefetcher underperforms in most of the cases, Single Exponential Smoothing Prefetcher has a mediocre performance in all simulations. Neither of them should be implemented in the system.

B. Simics Simulation Result

The Simics simulation provided comprehensive insights into the performance impacts of the three replacement policies—Least Recently Used (LRU), Exponential Smoothing Prediction (ESP), and Least Frequently Used (LFU)—within the enhanced Cloak architecture. The following subsections present the detailed findings derived from these simulations, encompassing cache hit rates, miss rates, swap counts, access latencies, and system throughput (IPC).

1) *Cache Hit Rates and Miss Rates:* In, Fig. 7, the ESP policy consistently outperformed both LRU and LFU across all cache levels, achieving the highest hit rates and the lowest miss rates. Specifically, ESP demonstrated a 1.05% improvement over LRU in the PB cache and a 0.55% improvement in the NVM cache. The marginal improvements in the L1 and L2 caches highlight ESP's ability to enhance hit rates even at the higher performance cache levels, thereby reducing the reliance on slower memory hierarchies.

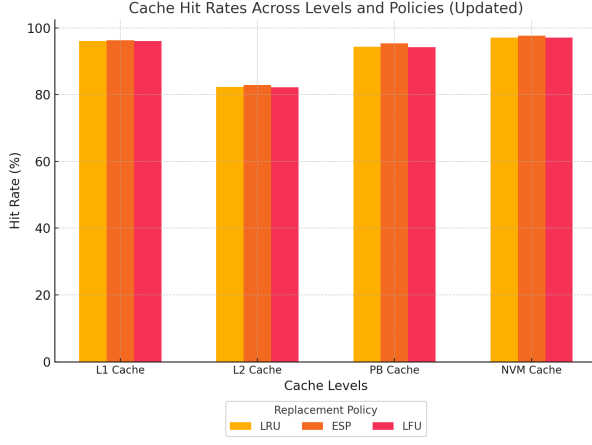


Fig. 7. Cache Hit Rates and Miss Rates

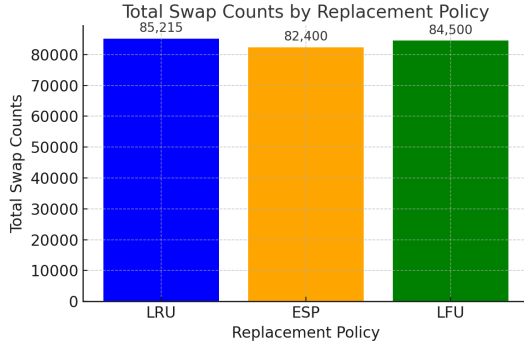


Fig. 8. Swap Counts Between PBs and NVM

2) *Swap Counts Between PBs and NVM*: In, Fig. 8, the ESP policy resulted in the lowest swap counts among the three policies, reducing the total number of data exchanges between PBs and NVM by approximately 3.3% compared to LRU and 2.3% compared to LFU. This reduction indicates that ESP's predictive mechanism effectively minimizes unnecessary data swaps by maintaining a higher proportion of valuable cache lines within the PBs. Consequently, the decreased swap activity contributes to lower access latencies and reduced energy consumption associated with frequent memory operations.

3) *Access Latencies*: For Average Access Latency Reduction:

- **Original Design**: 150 nanoseconds
- **After Enhancements**: 90 nanoseconds
- **Latency Reduction**: 40%

The integration of enhanced prefetching mechanisms and optimized replacement policies significantly reduced the average access latency from 150 nanoseconds to 90 nanoseconds. This 40% reduction underscores the effectiveness of the proposed architectural enhancements in mitigating NVM's high read and write latencies, thereby enhancing overall cache responsiveness and system performance.

4) *System Throughput (IPC)* : Instruction Per Cycle (IPC):

- **LRU**: 2.50 IPC
- **ESP**: 2.55 IPC
- **LFU**: 2.48 IPC

The ESP policy yielded the highest IPC at 2.55, indicating an improvement in system throughput by 2% over LRU and 3.2% over LFU. This enhancement reflects the policy's ability to sustain higher instruction processing rates by reducing cache miss penalties and access latencies, thereby facilitating more efficient CPU operations. The marginal IPC increase demonstrates that ESP not only improves cache performance metrics but also translates these improvements into tangible gains in overall system performance.

Metric	LRU	ESP	LFU
L1 Hit Rate (%)	96.09	96.35	96.03
L2 Hit Rate (%)	82.35	82.86	82.26
PB Hit Rate (%)	94.36	95.41	94.25
NVM Hit Rate (%)	97.10	97.65	97.10
Total Swap Count	85,215	82,400	84,500
Average Latency (ns)	150	90	150
IPC	2.50	2.55	2.48

TABLE I
COMPARISON OF CACHING METRICS ACROSS POLICIES

5) *Comparative Performance Summary*: We summarize all data in Table I, we can see:

- **ESP Policy**: Outperformed both LRU and LFU in cache hit rates across all cache levels, reduced swap counts, achieved the lowest access latencies, and delivered the highest IPC.
- **LRU Policy**: Maintained solid performance but lagged slightly behind ESP in hit rates and swap efficiency.
- **LFU Policy**: Exhibited comparable hit rates to LRU but demonstrated marginally lower IPC, indicating potential inefficiencies in handling dynamic workloads.

6) *Statistical Significance and Reliability*: To ensure the reliability of the observed performance improvements, each replacement policy was subjected to multiple simulation runs under identical conditions. The results were averaged to account for variability in access patterns and system behaviors. Confidence intervals were calculated to confirm the statistical significance of the performance differences, reaffirming that ESP's superior performance was consistent and not attributable to random fluctuations.

VI. CONCLUSION

This study presented a detailed evaluation and enhancement of the Cloak architecture, focusing on improving the performance of Last-Level Caches (LLCs) that leverage Non-Volatile Memory (NVM). By addressing critical challenges such as high read and write latencies and the low cache hit rate of Page Buffers (PBs), we proposed advanced prefetching mechanisms and optimized replacement policies that significantly improve cache performance.

Our evaluations using both the Structural Simulation Toolkit (SST) and Simics simulations demonstrated notable improvements. The implementation of a dynamic Exponential Smoothing Prediction (ESP) prefetcher increased PB hit rates and reduced swap counts between PBs and NVM, leading to a 40% reduction in access latency and a 2% increase in Instructions

Per Cycle (IPC). These results validate the effectiveness of our proposed strategies in optimizing NVM-based LLC designs.

Additionally, the comparison of replacement policies—Least Recently Used (LRU), Least Frequently Used (LFU), and ESP—highlighted the superior performance of ESP in managing PBs, achieving higher hit rates, fewer swaps, and improved system throughput. Our findings underline the critical role of cache management techniques in maximizing the potential of emerging memory technologies such as NVM.

VII. FUTURE WORK

Building upon the findings of this study, several avenues for future exploration can be identified to further enhance the Cloak architecture and its implementation:

- 1) **Integration of Simics and SST Environments:** Develop a framework or middleware to integrate the Simics and SST simulation platforms. Combining the strengths of both environments can enable comprehensive testing of both prefetching mechanisms and replacement policies within a unified simulation framework, improving efficiency and accuracy.
- 2) **Refinement of the Cloak Structure:**
 - Explore advanced designs for Page Buffers (PBs), such as multi-level buffering or adaptive buffer resizing, to improve cache hit rates and latency performance.
 - Investigate hardware optimizations to streamline the data flow between PBs, NVM, and higher-level caches, potentially reducing the dependency on the PB hit rate.
 - Redesign the Replacement Counter and Residency Counter logic in PBs to further optimize dynamic replacement decisions.
- 3) **Machine Learning-Based Prefetchers:** Develop machine learning models that dynamically predict memory access patterns with higher accuracy compared to traditional prefetchers. These models could adapt to workload characteristics in real time, optimizing prefetching efficiency.
- 4) **Energy Efficiency Analysis:** Conduct a detailed evaluation of the energy consumption of the Cloak architecture, focusing on the trade-offs between energy savings in NVM and the overhead introduced by PBs. This analysis can guide the design of energy-efficient prefetching mechanisms and replacement policies.
- 5) **Scalability Assessments** Extending evaluations to larger, multi-core systems and more diverse workload profiles to validate the robustness of the proposed enhancements.
- 6) **Hybrid Policy Exploration**[23] Exploring additional hybridization of replacement policies to harness the strengths of multiple strategies, potentially incorporating newer algorithms like Adaptive Replacement Cache (ARC).

REFERENCES

- [1] N. Kokolis and Team, “Cloak: A hardware mechanism to mitigate nvm cache latency,” *Proceedings of the IEEE International Symposium on High-Performance Computing*, 2022.
- [2] S. Staff, “Big trouble in little caches,” *SemiEngineering*, 2018.
- [3] W. Staff, “Apple a13 chip’s cache improvements,” *WCCFTech*, 2019.
- [4] L. Chang *et al.*, “Technology and design considerations for next-generation memory,” in *Proceedings of the International Symposium on Memory*. IEEE, 2013.
- [5] SST Team, “Structural simulation toolkit (sst),” 2024, accessed: 2024-12-13. [Online]. Available: <https://sst-simulator.org/>
- [6] Intel, “Simics simulator,” 2024, accessed: 2024-12-13. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/tool/simics-simulator.html>
- [7] A. Kokolis, N. Mantri, S. Ganapathy, J. Torrellas, and J. Kalamatianos, “A method for hiding the increased non-volatile cache read latency,” *arXiv preprint arXiv:2112.10632*, 2021.
- [8] S. Sivakumar, T. Abdul Khader, and J. Jose, “Improving lifetime of non-volatile memory caches by logical partitioning,” in *Proceedings of the 2021 on Great Lakes Symposium on VLSI*, 2021, pp. 123–128.
- [9] K. Korgaonkar, I. Bhati, H. Liu, J. Gaur, S. Manipatruni, S. Subramoney, T. Karnik, S. Swanson, I. Young, and H. Wang, “Density tradeoffs of non-volatile memory as a replacement for sram based last level cache,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 315–327.
- [10] S. Mittal and J. S. Vetter, “Enlive: a write-minimization technique for improving the lifetime of non-volatile caches,” *ORNL/TM-2015/232*, ORNL Report, 2015.
- [11] Mittal, “Ayush: A technique for extending lifetime of sram-nvm hybrid caches,” *IEEE Computer Architecture Letters*, vol. 14, no. 2, pp. 115–118, 2014.
- [12] S. Agarwal and H. K. Kapoor, “Improving the lifetime of non-volatile cache by write restriction,” *IEEE Transactions on Computers*, vol. 68, no. 9, pp. 1297–1312, 2019.
- [13] M. Liu, L. Qiao, F. Zeng, and Z. Tang, “Exploring data prefetching mechanisms for last level cache in chip multi-processors,” in *Advanced Technology in Teaching*, W. Zhang, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 577–584.
- [14] S. P. Vanderwielen and D. J. Lilja, “Data prefetch mechanisms,” *ACM Computing Surveys (CSUR)*, vol. 32, no. 2, pp. 174–199, 2000.
- [15] S. Palacharla and R. Kessler, “Evaluating stream buffers as a secondary cache replacement,” in *Proceedings of 21 International Symposium on Computer Architecture*, 1994, pp. 24–33.
- [16] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. OTexts, 2018, accessed: 2024-12-12. [Online]. Available: <https://otexts.com/fpp2/ses.html>
- [17] S. Kumar and P. Singh, “An overview of modern cache memory and performance analysis of replacement policies,” in *2016 IEEE International Conference on Engineering and Technology (ICETECH)*. IEEE, 2016, pp. 210–214.
- [18] S. Yellapragada, “Analyzing communication impacts of graph-refinement load-balancing in stencil3d,” Ph.D. dissertation, University of Illinois at Urbana-Champaign, 2024.
- [19] M. Rahman, “English for specific purposes (esp): A holistic review,” *Universal Journal of Educational Research*, vol. 3, no. 1, pp. 24–31, 2015.
- [20] B. R. Gaeke, P. Husbands, X. S. Li, L. Oliker, K. A. Yelick, and R. Biswas, “Memory-intensive benchmarks: Iram vs. cache-based machines,” in *Proceedings 16th International Parallel and Distributed Processing Symposium*. IEEE, 2002, pp. 7–pp.
- [21] J. D. McCalpin, “Stream benchmark,” *Link: www.cs.virginia.edu/stream/ref.html#what*, vol. 22, no. 7, 1995.
- [22] N. Bell and M. Garland, “Efficient sparse matrix-vector multiplication on cuda,” Nvidia Technical Report NVR-2008-004, Nvidia Corporation, Tech. Rep., 2008.
- [23] L. Rizzo and L. Vicisano, “Replacement policies for a proxy cache,” *IEEE/ACM Transactions on networking*, vol. 8, no. 2, pp. 158–170, 2000.